

Implementation of HEPT algorithm for CMS data

Evgueni Alexeev

7/15/25

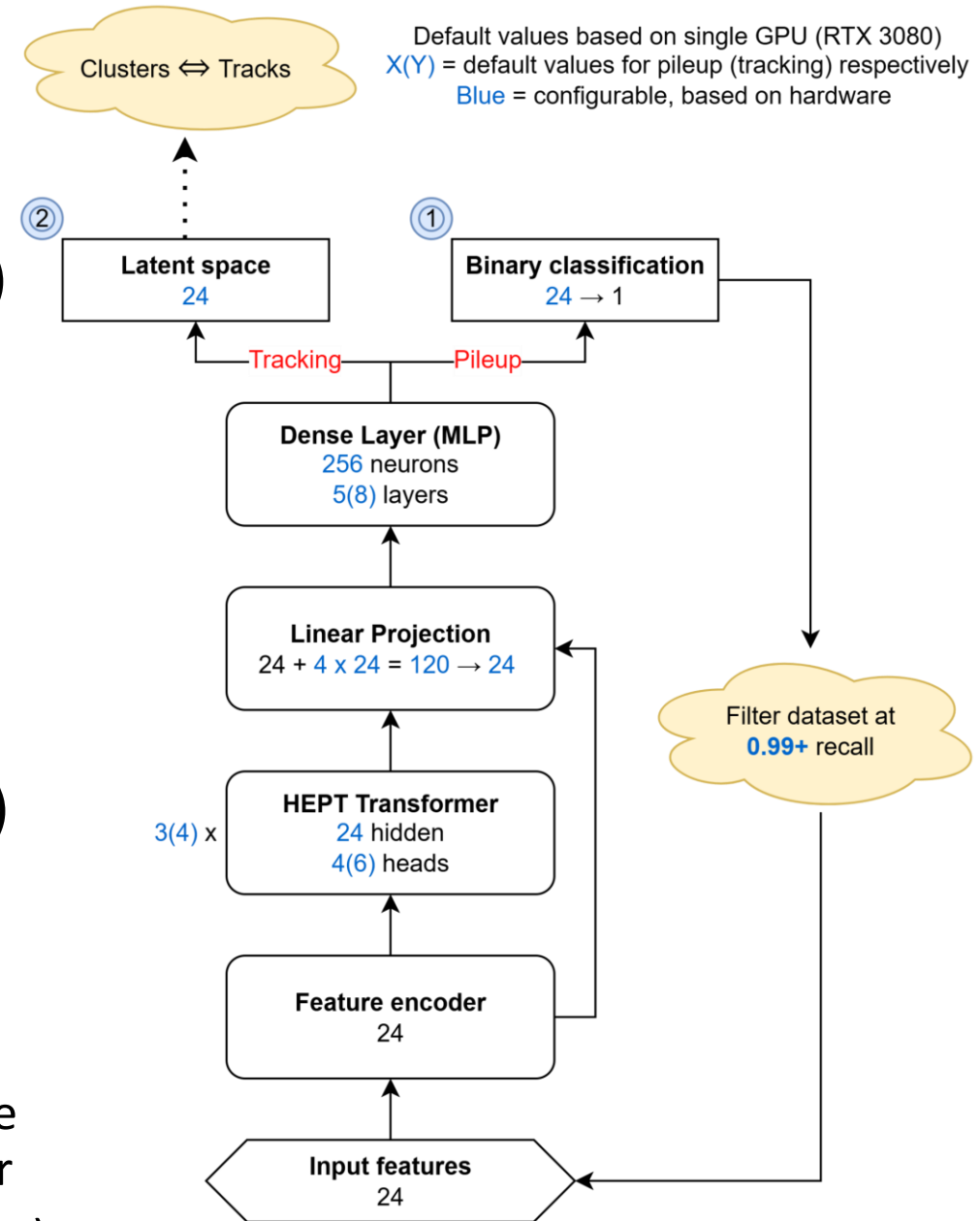
Based on <https://arxiv.org/abs/2402.12535>

Locality-Sensitive **H**ashing-Based **E**fficient **P**oint **T**ransformer with Applications in High-Energy Physics

Siqi Miao, Zhiyuan Lu, Mia Liu, Javier Duarte, Pan Li

Pipeline

- Input features: all LST constructed line segments (LS)
 - $\sin(\phi), \cos(\phi), \eta, p_t$
 - $r, x, y, z, d\phi$ (4-hit average)
 - intra-MD $\Delta r, \Delta x, \Delta y, \Delta z, \Delta d\phi$
 - inter-MD dr, dx, dy, dz
 - Start & end layers
- Initially **93%** fake/**7%** true (*LS_isFake* label)
- Input and train '**pileup**' model (here pileup = fake LS)
 - Binary classification** using Focal Loss
 - Make cut based on recall target (used **99% recall**)
- Use reduced dataset to train '**tracking**' model
 - Output: **24-dim embedding space**, where LS from the same sim track (*LS_SimTrkIdx* label) are clustered closer together
 - No additional post-processing (in principle can do clustering)



More detail

- **Pileup** model uses **Focal Loss**

- Additionally adds class weights to counteract class imbalance
- Uses $\gamma = 2$, dynamically adjusts class weights based on each event, additionally overtuned (2x) to prioritize recall (true LS)
- AUCPR and F_β -score, with $\beta = 3$ to again emphasize recall

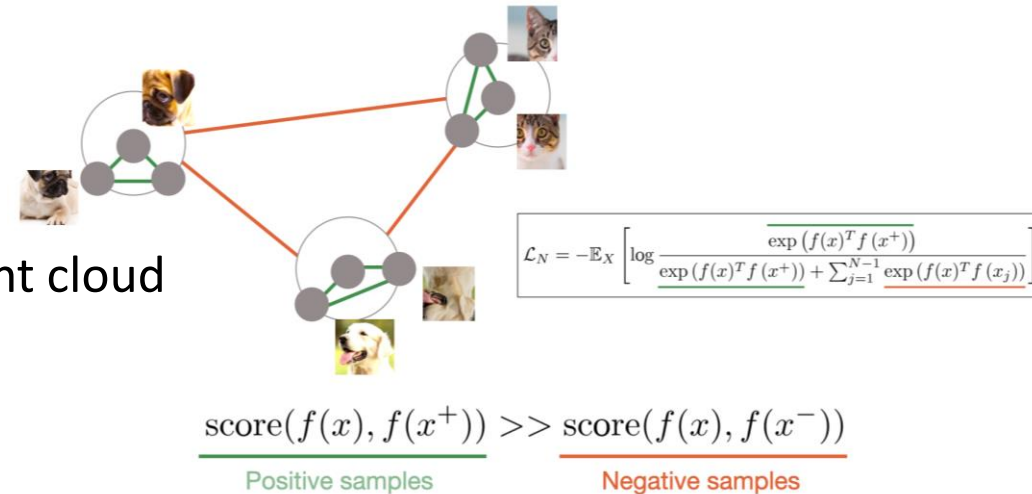
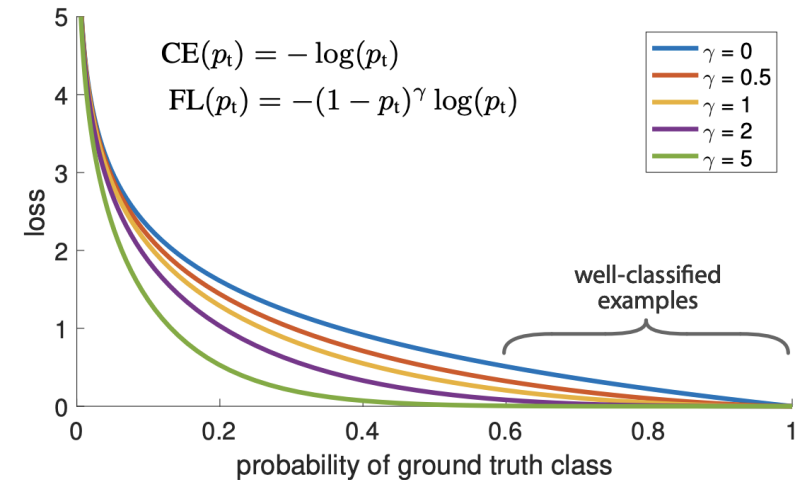
$$F_\beta = \frac{(1 + \beta^2) \cdot \text{TP}}{(1 + \beta^2) \cdot \text{TP} + \beta^2 \cdot \text{FN} + \text{FP}}$$

- **Tracking** model uses **InfoNCE** (contrastive loss)

- Positive edges connect two LS in the same track
- Negative edges are NN connections to other LS in the point cloud
- Evaluated using Average Precision @ k score

$$\text{AP@k} = \frac{1}{N} \sum_{i=1}^N \text{Prec @ } k_i$$

- For each LS, find sim track index and check how many of its $k = 1, 2, \dots, L-1$ nearest neighbors in the latent space belong to the same sim track, and average across k and across all LS



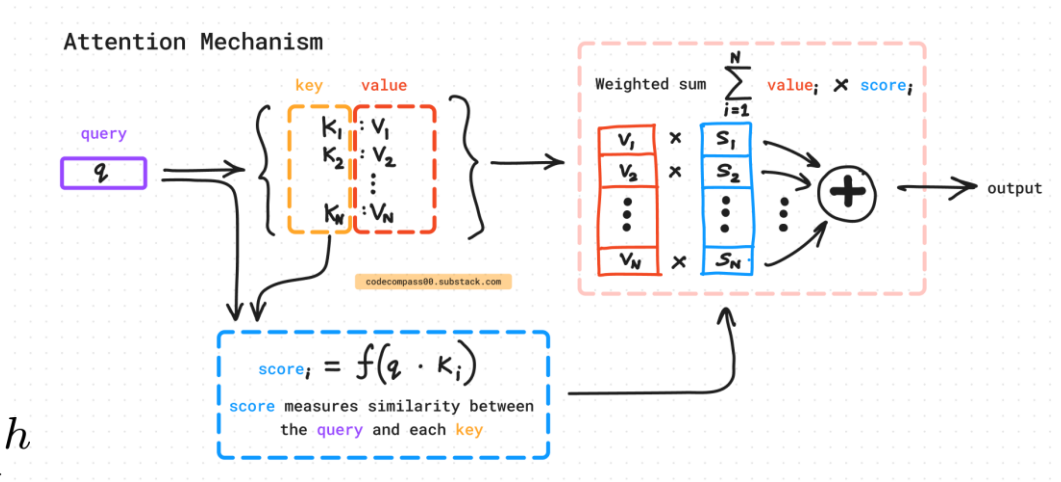
Transformer attention

- Self-attention mechanism

- Each LS ('point') has 3 vectors: $q_x, k_x, v_x \in \mathbb{R}^h$
- h = embedding dimension (in our case, default is $h = 24$)
- Stacking N points we have 3 matrices: $Q, K, V \in \mathbb{R}^{h \times N}$
- Q, K, V are a set of learned weight matrices, which capture **pairwise relations** between points through **attention mechanism**:

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- Attention: $x_i \rightarrow A(x_1, \dots, x_N)$, gives weighted sum of all points based on similarity
- Computational complexity of $\mathcal{O}(N^2)$
 - Same as fully-connected graph – cost prohibitive for high pileup data
 - HEPT uses **LSH** to **approximate** Attn to reduce this to $\mathcal{O}(N \log N)$



Efficient transformer

- Main idea: points/LS from the same track will be close together in some coordinate space (in orig paper – taken to be ϕ, η)
 - Points need to attend only to other points nearby in this space
 - Replace dot-product qk^T with **Euclidean distance**:
$$d(q_i, k_j) = \exp\left(-\frac{1}{2}|q_i - k_j|^2\right)$$
 - Attention score goes to 0 as $d(q_i, k_j)$ increases
- Can approximate this distance via Euclidean Locality-Sensitive Hashing ([LSH](#))
 - LSH hashes close data points into the same bucket with high probability
 - For pairs of points in the **same bucket** → compute **full attention**

Locality-Sensitive Hashing-Based Efficient Point Transformer with Applications in High-Energy Physics

Siqi Miao¹ Zhiyuan Lu² Mia Liu³ Javier Duarte⁴ Pan Li¹

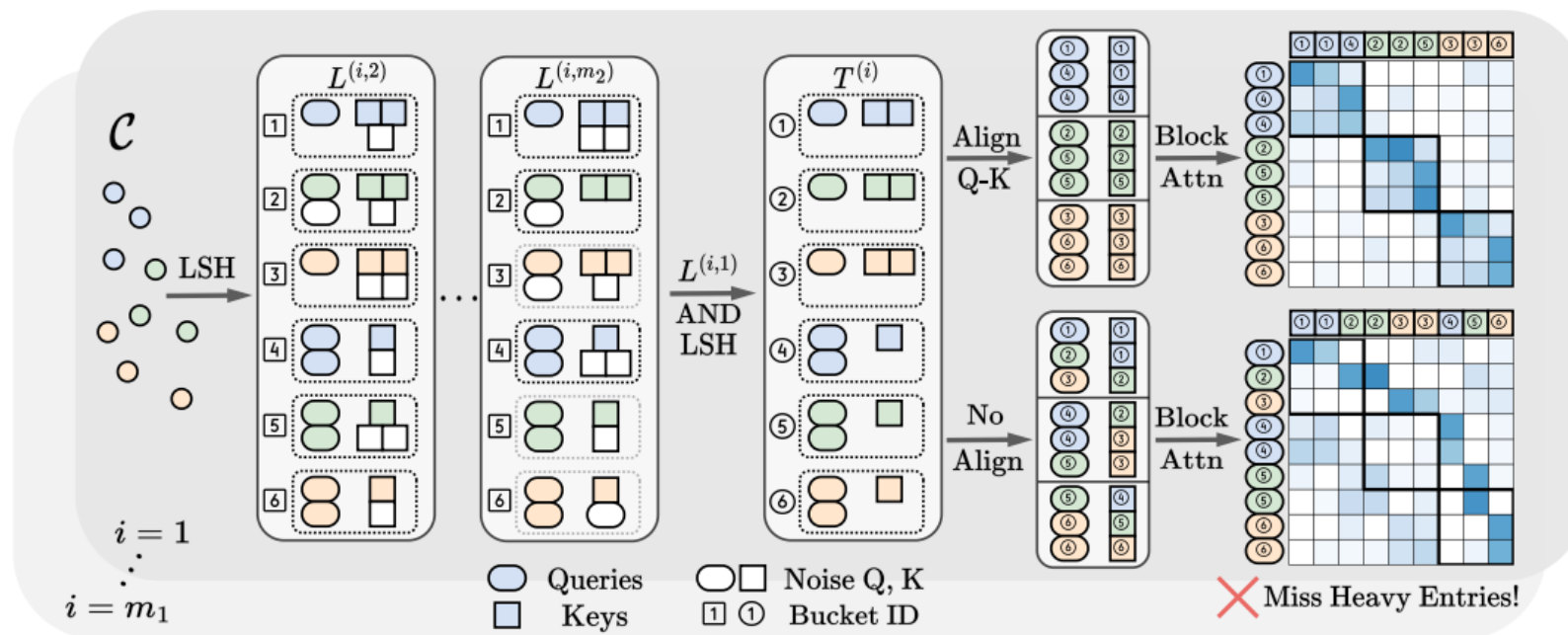


Figure 1: Pipeline of HEPT. Elements that share the same color represent points from the same local neighborhood. HEPT employs OR & AND LSH to minimize noise caused by individual hash functions. HEPT also integrates point coordinates as extra AND LSH codes for query-key alignment, maintaining computational regularity without compromising accuracy.

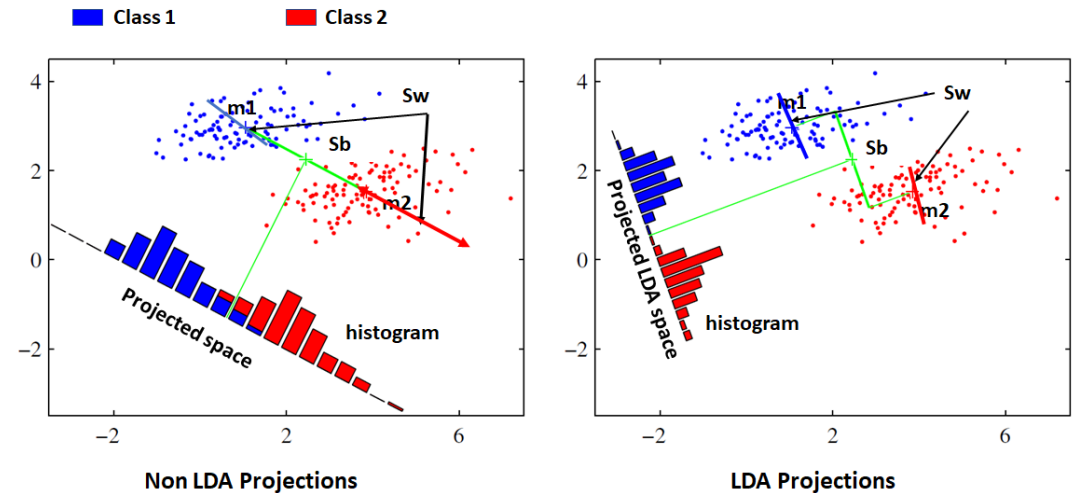
Linear Discriminant Analysis (LDA)

- First introduce $\phi \rightarrow \sin \phi, \cos \phi$ to solve periodicity issue
- Use LDA to find **linear combination of features** that max variance *between* clusters, and min variance *within* clusters

Each class has mean μ_i and same covariance C

$$\Sigma_b = \frac{1}{C} \sum_{i=1}^C (\mu_i - \mu) (\mu_i - \mu)^T$$

$$\text{Maximize 'SNR': } S = \frac{\sigma_{\text{between}}^2}{\sigma_{\text{within}}^2} = \frac{\vec{w}^T \Sigma_b \vec{w}}{\vec{w}^T \Sigma \vec{w}}$$

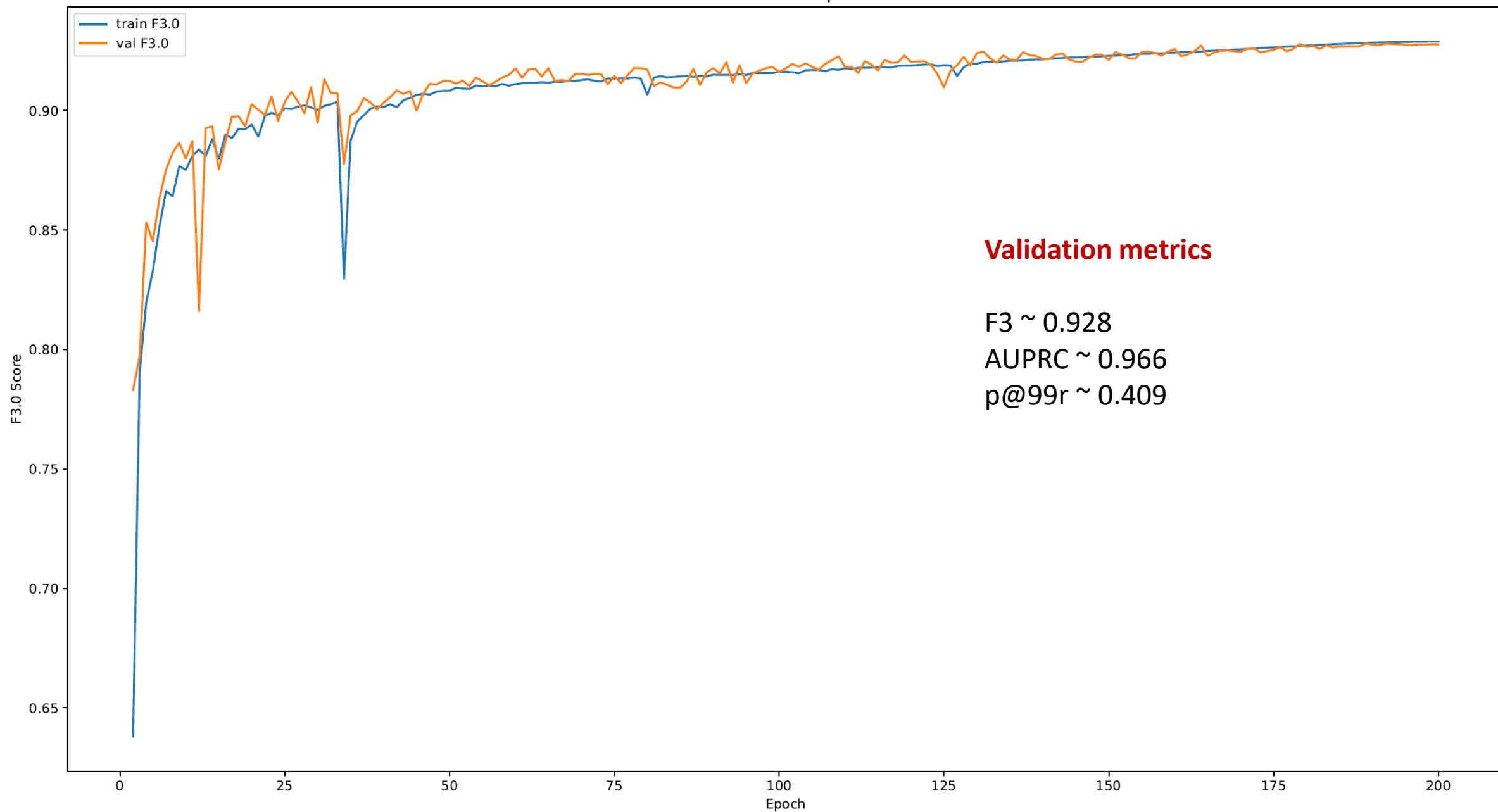


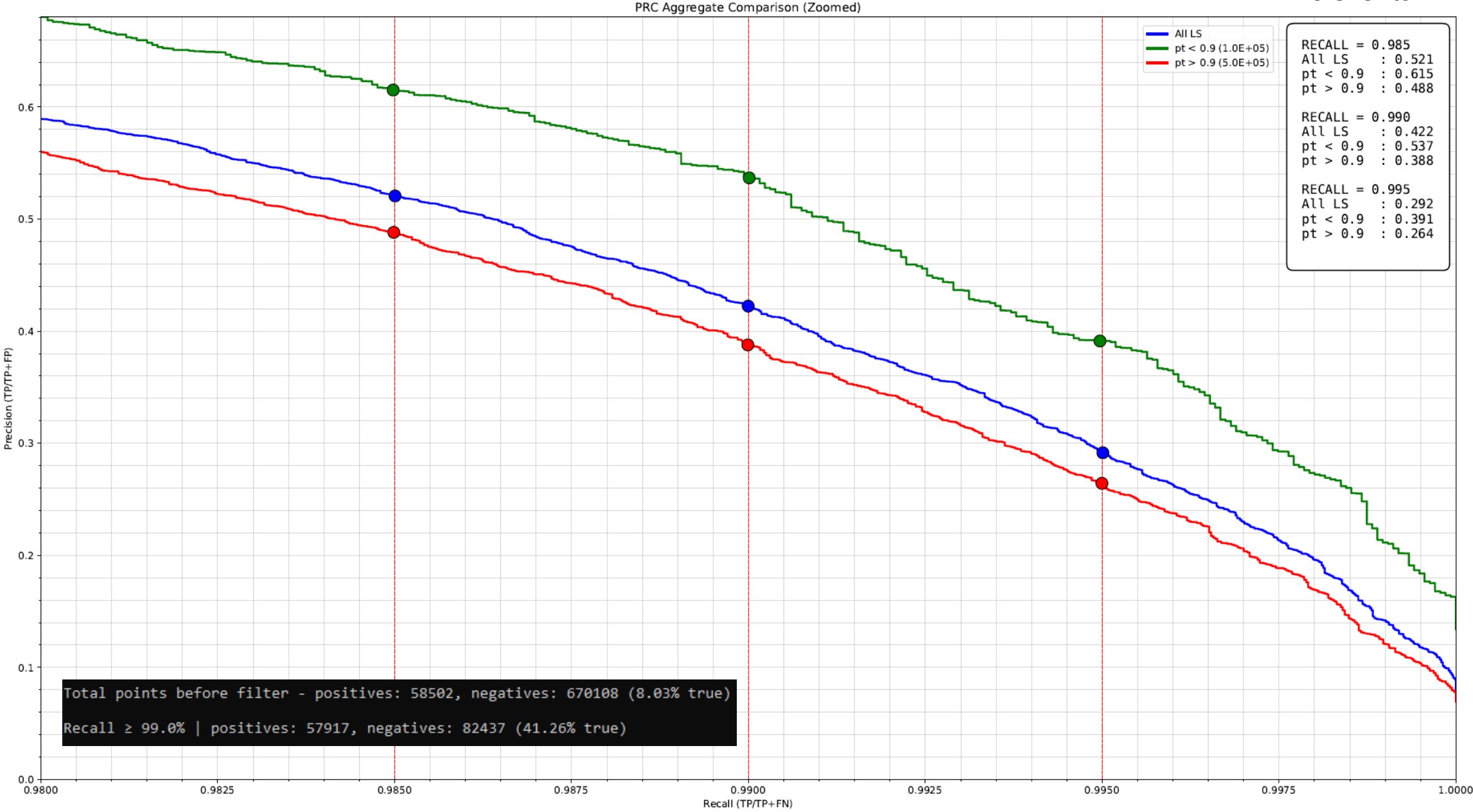
- Instead of using coordinates (η , $\sin \phi$, $\cos \phi$) in LSH AND buckets, use **top 3 LDA-transformed features** (**$\sim 99\%$** of variance captured)
- Saturates clusters with much fewer NN, lower noise contamination

Pileup Results

2000 events, 200 epochs on single A40 (40GB)
pu200 ntuple

F3.0 Score vs Epoch

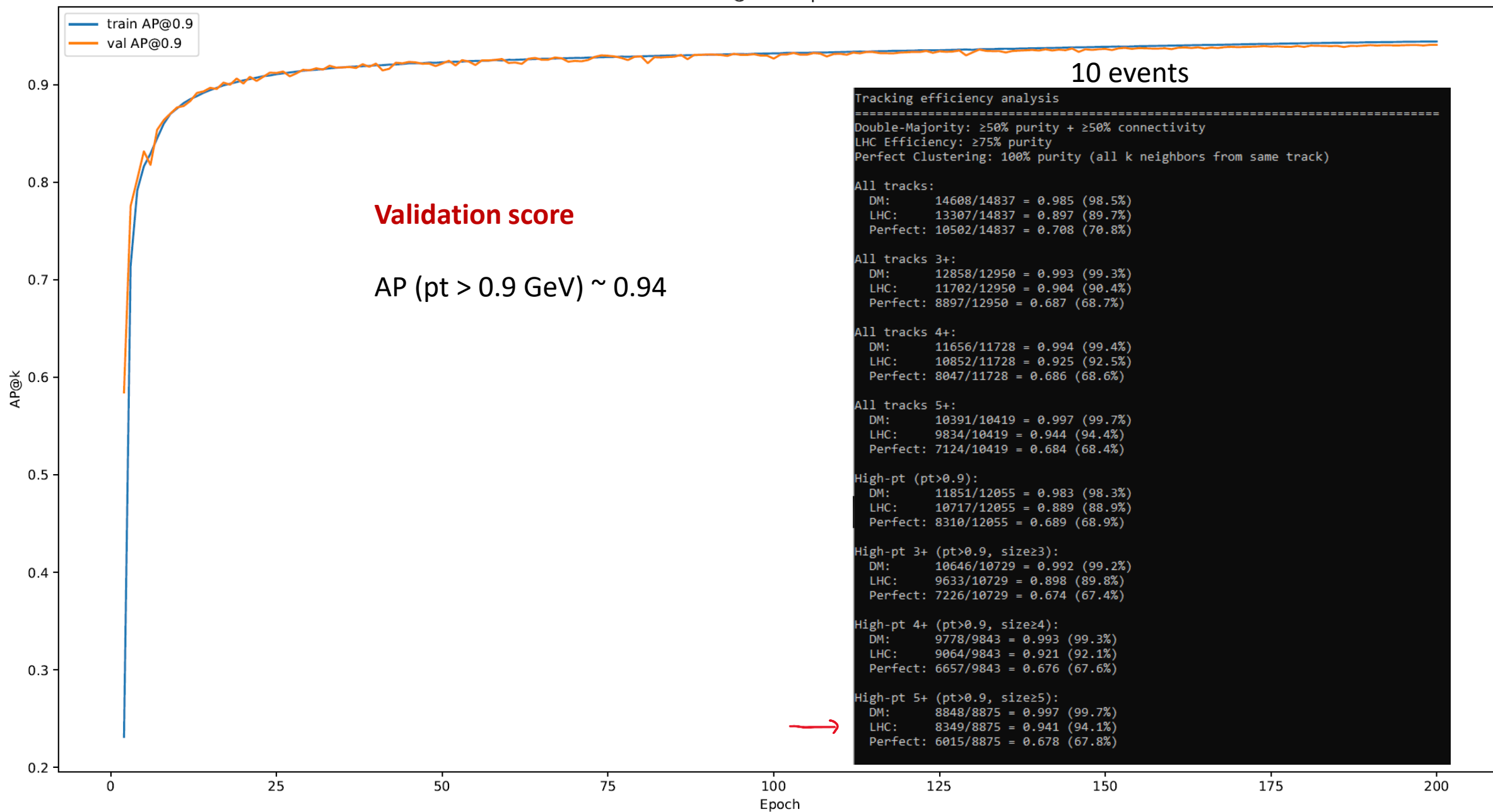


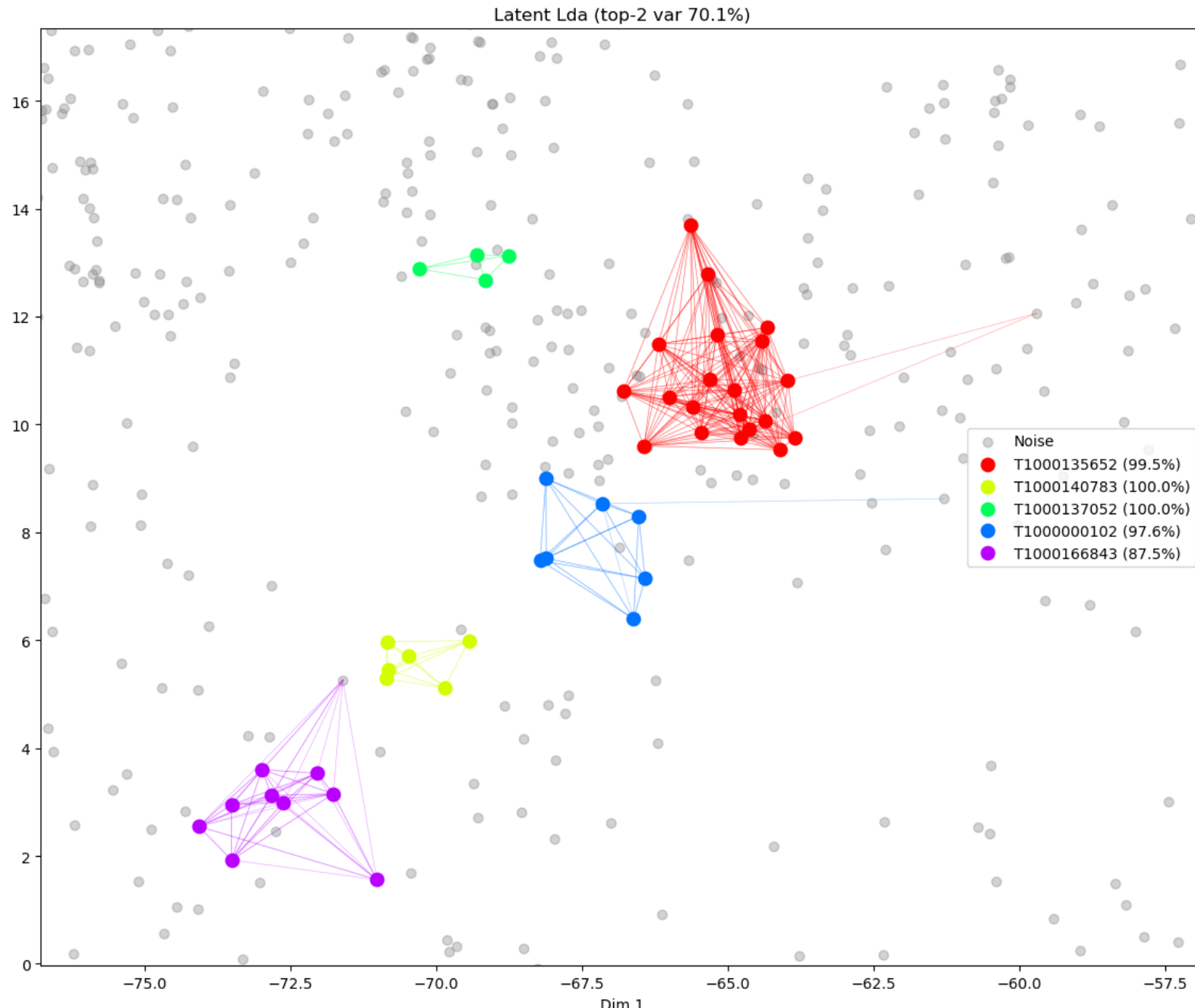


Tracking Results

2000 events, 200 epochs on single A40 (40GB)
pu200 ntuple

AP@0.9 vs Epoch



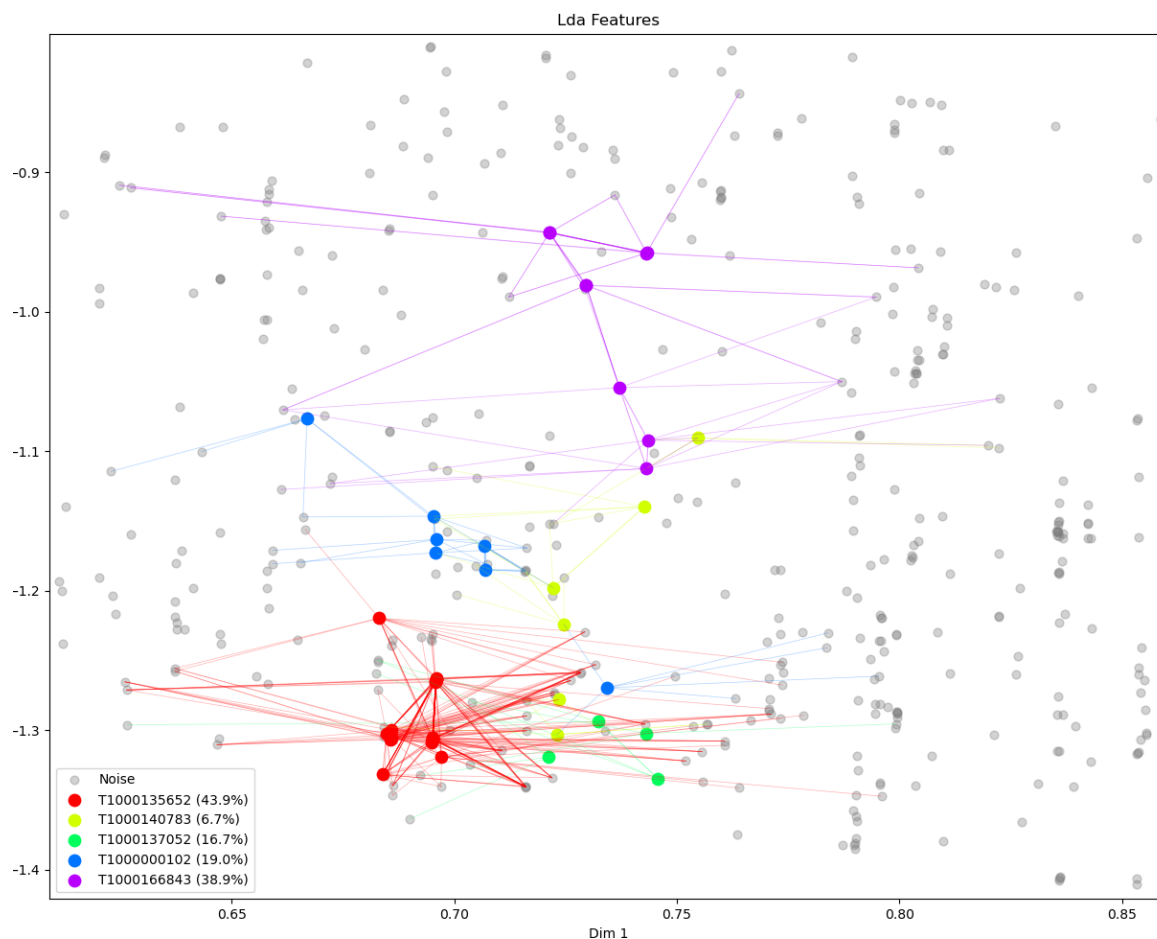
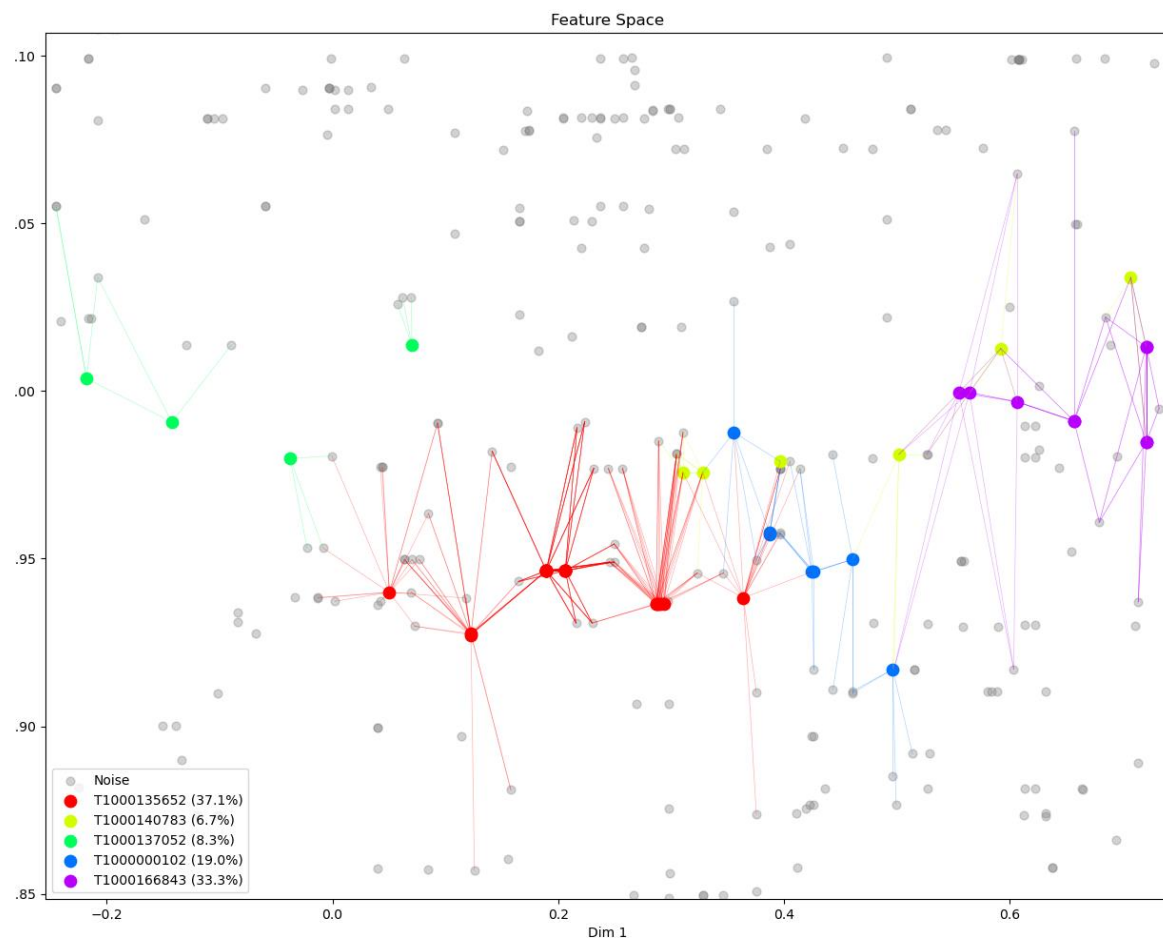


Note: Plot visualizes top 2 LDA components in output latent space, but efficiency values (legend) and connections are in *full 24d latent space* using Euclidean distance to find knn

- **Purity** - how many of L-1 nearest neighbors for each LS is from the same track, averaged over all LS in track
- **Efficiency (LHC)** - for each track, if more than 75% of points have more than 75% purity – track found, else not.

vs $\eta, \sin \phi, \cos \phi$

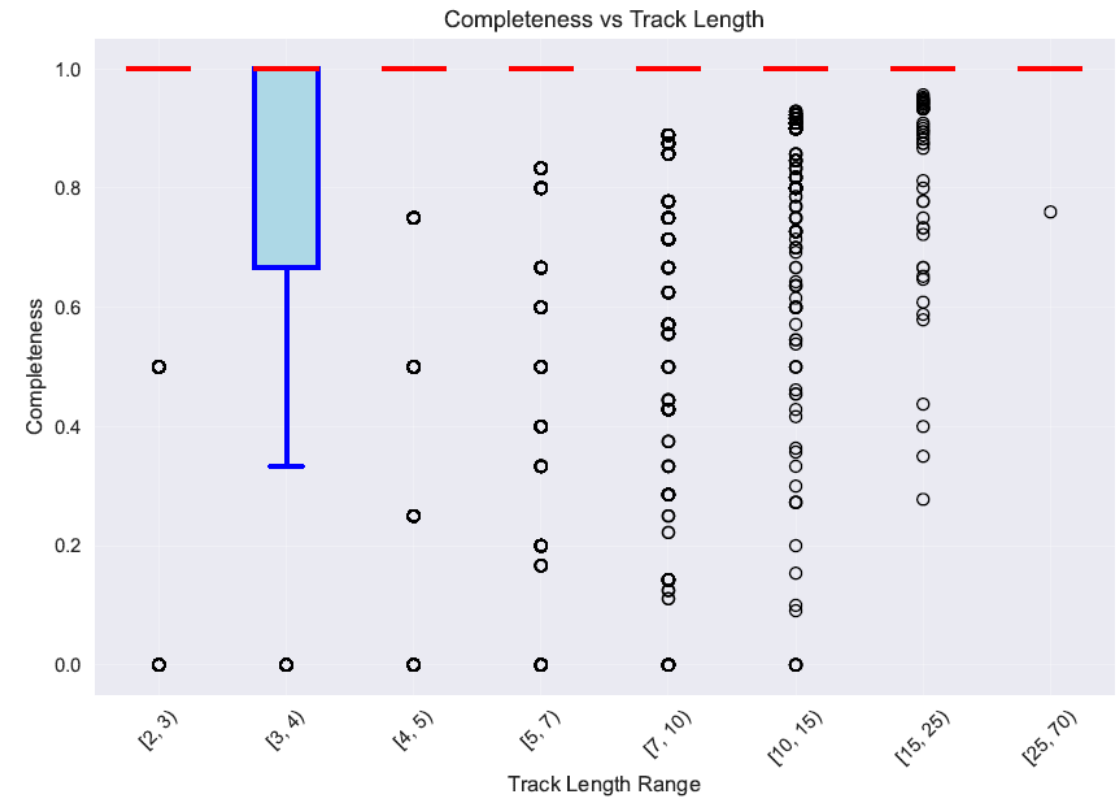
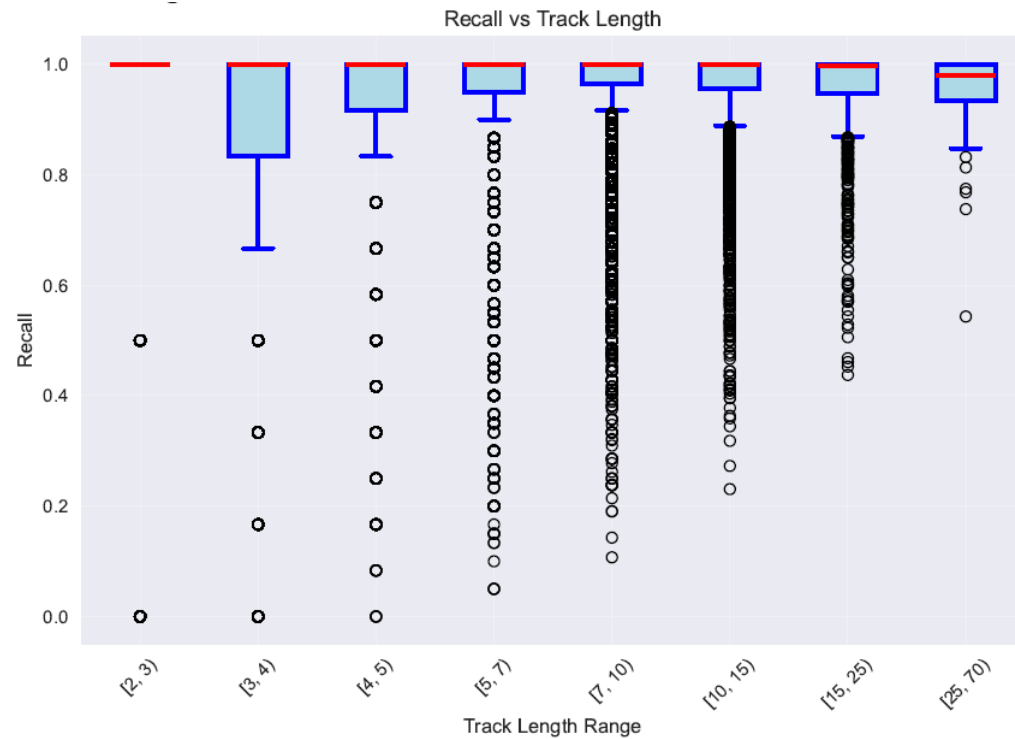
original LDA coords (top 3)



Model does better on longer tracks

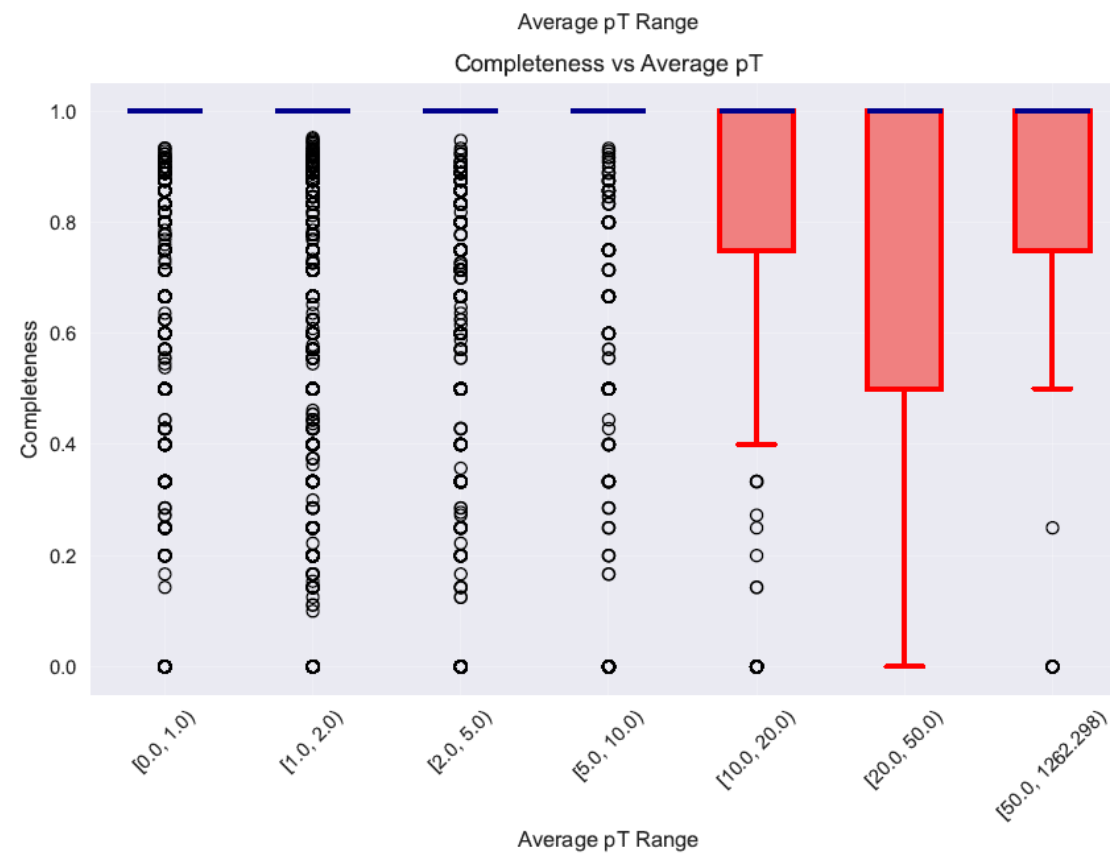
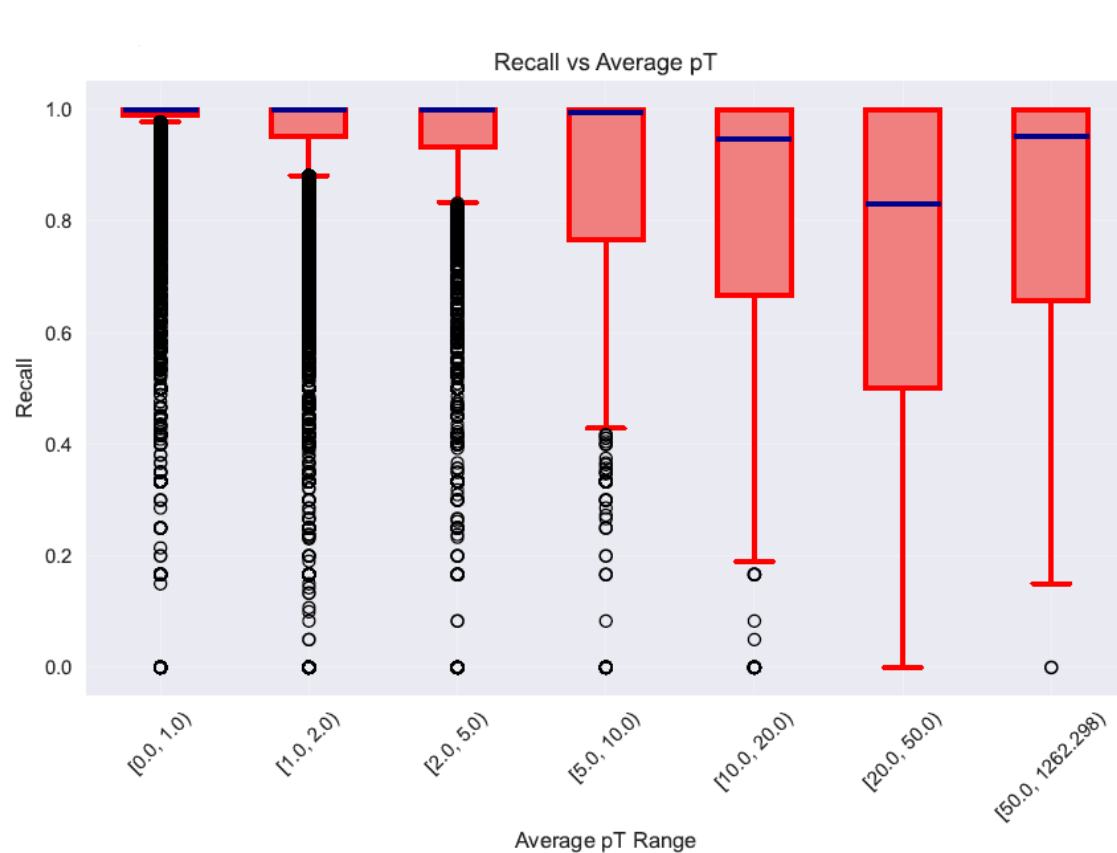
recall			
length_bin	mean	std	count
[2, 3)	0.889	0.282	1887
[3, 4)	0.874	0.224	1222
[4, 5)	0.914	0.169	1309
[5, 7)	0.950	0.112	4057
[7, 10)	0.959	0.091	4133
[10, 15)	0.962	0.079	1903
[15, 25)	0.964	0.068	316
[25, 44)	0.981	0.025	10

10 events



Model does better on smaller pt

10 events

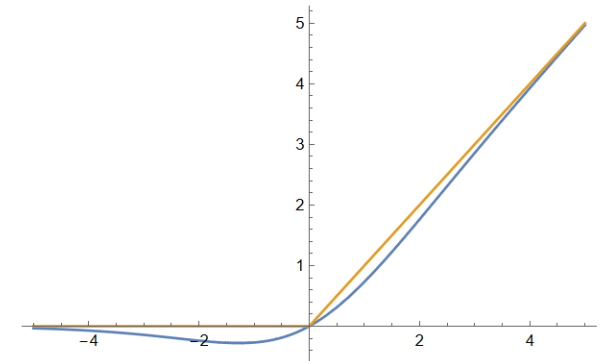


Additional info & changes from HEPT

- Flexible setup using Pytorch Lightning – single/batched events, CPU or single/multi GPU using **FSDP**
- Various optimizers available, **AdamW** gave best performance
- Used **SiLU**/Swish instead of ReLU for better perf:

$$x \cdot \sigma(x)$$

- Modified InfoNCE to **downweigh longer tracks**
- Modified tracking nn.radius_graph construction to assign fake/noise LS “fake” track ids, to push them away
- Used **cosine** learning rate schedule with warmup



<https://github.com/evgueni-alexeev/HEPT-CMS>